COMPLETE TECHNICAL DOCUMENTATION

THE TRACKER

Track. Improve. Repeat.

Next.js 16.2.7 | React 19 | TypeScript 5 | Supabase (PostgreSQL) | Tailwind CSS 4 | Google OAuth 2.0 | Vercel

---

Document Type: Complete Technical Documentation & Architecture Guide

Version: 1.0.0 · Generated: June 2026

Author: Hussein A-H · Repository: github.com/HusseinA-H/THE-TRACKER

Classification: Portfolio / Engineering Reference

Sections: 42 · Diagrams: 24 · Tables: 18

Table of Contents

PART V — ARCHITECTURE

PART VI — ENGINEERING

SECTION 01

Executive Summary

THE TRACKER is a purpose-built personal athlete performance platform designed and engineered for a specific Upper & Lower hypertrophy split training methodology. It replaces three generic fitness apps (Strong, Hevy, Boostcamp) with a single, fully owned, subscription-free ecosystem.

24	11	12	6	3	0
Production Routes	SQL Migrations	Database Tables	Workout Templates	RBAC Roles	Custom API Routes

Build Status: Passing

All 24 routes compile without TypeScript errors. Production build verified on Next.js 16 with Turbopack. RLS enforced at the PostgreSQL layer across all 12 tables.

Key Decisions

DECISION	CHOICE	RATIONALE
API Strategy	Next.js Server Actions (zero custom API routes)	Eliminates API layer boilerplate, co-locates mutations with features
Auth Provider	Supabase Auth (Google OAuth 2.0)	One-click sign-in, automatic JWT management, cookie-based sessions
Data Security	PostgreSQL Row-Level Security on all tables	Database-enforced isolation; cannot be bypassed by application code
Styling	Tailwind CSS v4 + shadcn/ui primitives	Utility-first, zero runtime CSS, fully accessible component base
Deployment	Vercel (Edge) + Supabase Cloud	Global CDN, zero-config Next.js deployment, managed PostgreSQL

SECTION 02

Product Overview

THE TRACKER is not a SaaS product. It is a personal training ecosystem engineered around one specific athlete's program. The platform models a 10-day rolling training cycle and provides every tool needed to execute, track, and progressively overload a structured hypertrophy program.

Exercise Library

Global + custom exercises with muscle group classification, video references, and alternative swap configurations.

Workout Templates

6 predefined sessions: Upper A/B/C, Lower A/B, Cardio+Abs. Each contains ordered exercises with target sets and rep ranges.

Live Workout Logger

Active session tracker with real-time progression hints, PR detection, rest timer, and set type classification.

PR System

Auto-tracks max weight, max volume, and estimated 1RM. Fires gold badge overlay in real time when a PR is broken.

Training Calendar

10-day rolling cycle mapped to a calendar. Shows completed, missed, and rest days. Configurable start date.

Admin Dashboard

RBAC-protected management panel for exercises, templates, packages, alternatives, videos, and users.

SECTION 03

Business Problem

Existing fitness apps are designed for general audiences. Athletes following structured periodized programs face fundamental constraints that no mainstream app addresses.

PROBLEM	APPS AFFECTED	IMPACT ON ATHLETE
No support for non-weekly rolling cycles	Strong, Hevy, Boostcamp	Cannot model a 10-day Upper/Lower/Rest rotation

PROBLEM	APPS AFFECTED	IMPACT ON ATHLETE
No automatic progression suggestions	Strong, MyFitnessPal	Must manually recall last weights from memory
Generic exercise database (no alternatives)	All major apps	Cannot configure equipment substitutions mid-workout
Subscription-gated analytics	Hevy, Boostcamp	Key performance data behind $10–20/month paywalls
No template-to-workout auto-population	Most apps	Must re-enter every exercise and set each session
No admin-controlled exercise library	All apps	Cannot enforce a curated, coach-controlled movement list

💡 **The Core Insight**

The best tracking tool is the one built for you. A personal platform with zero compromises eliminates the friction that prevents consistent data entry — which is the actual limiting factor for most athletes, not motivation.

SECTION 04

Product Vision

THE TRACKER is designed to feel like a combination of three industry-leading apps, fully customized for a single athlete's methodology.

APP	WHAT IT DOES BEST	WHAT THE TRACKER ADDS
Strong	Clean, frictionless workout logging	Custom template auto-population + cycle tracking
Hevy	Exercise history & PR visualization	Real-time PR detection + progression suggestions
Boostcamp	Coach-prescribed program execution	Fully customizable — owned by the athlete

Ecosystem Map

Exercises
Library of movements with muscle groups, videos, alternatives

Templates
6 predefined sessions, extensible

Packages
Group templates into named programs

Sessions
Live logging with progression

Weight
Daily body weight tracking + charts

PRs
Auto-tracked personal records

Calendar
Rolling 10-day cycle visualization

Admin
Management + RBAC controls

SECTION 05

User Personas

The Athlete (Primary)
Follows a structured Upper/Lower hypertrophy program. Wants frictionless logging, automatic progression guidance, and performance history. Logs 4–6 sessions per 10-day cycle.

Pain Points: Forgetting last weights, re-entering exercises every session, no PR visibility.

Goals: Log sessions in under 2

The Admin (Secondary)
Manages the exercise library, workout templates, and packages. Curates alternatives and video references. Assigns roles to users.

Pain Points: No centralised tool to manage shared exercise data.

Goals: Maintain a clean, curated exercise library with coaching annotations.

Super Admin
Full system access including user management and role assignment. Can promote users to admin, configure system settings, and manage all platform data.

Permissions: All admin capabilities + role changes + system settings.

minutes, see progression, track PRs.

SECTION 06

Functional Requirements

Authentication & User Management

- ✓ Sign in with Google OAuth 2.0 (one-click, no password)
- ✓ Automatic profile creation via PostgreSQL trigger on first login
- ✓ Session persistence via Supabase JWT cookies
- ✓ Session refresh on every middleware-intercepted request
- ✓ Role-based access: user, admin, super_admin
- ✓ Redirect unauthenticated users to /login
- ✓ Redirect authenticated users away from /login

Exercise System

- ✓ Browse global exercise library (system-level, no user_id)
- ✓ Filter exercises by category (Upper / Lower / Cardio) and muscle group
- ✓ Search exercises by name
- ✓ Admin: Create, update, delete system exercises
- ✓ Admin: Assign video URL to exercises
- ✓ Admin: Configure up to 3 alternative exercises per movement (preference-ordered)
- ✓ Exercise detail drawer: Overview, History, Notes, Alternatives tabs

Workout System

- ✓ Display 6 predefined workout templates with name, muscle focus, exercise count, estimated duration
- ✓ Start a workout session from a template (auto-populates exercises and sets)
- ✓ Log sets with weight, reps, set type (warmup/working/top/failure), and notes

- ✓ View last weight and last reps for each exercise per set input
- ✓ Receive computed suggestion for next session weight
- ✓ Built-in rest timer (default 90s, configurable)
- ✓ Add/remove exercises mid-session
- ✓ Swap an exercise for a configured alternative
- ✓ Finish workout with confirmation dialog
- ✓ Cancel workout with confirmation dialog
- ✓ View full workout history with session details

Tracking & Analytics

- ✓ Automatic PR detection during active workout (weight, volume, estimated 1RM)
- ✓ Real-time PR badge overlay when a new record is set
- ✓ Personal records table: max weight, max estimated 1RM, max volume per exercise
- ✓ PR history with date, type, value, and linked workout entry
- ✓ Daily body weight logging with date picker
- ✓ Weight trend chart (Recharts line graph)
- ✓ Body measurements logging: waist, chest, shoulders, arms, forearms, thighs, calves
- ✓ Bi-directional sync between weight_logs and body_measurements via PostgreSQL trigger
- ✓ Training calendar: month view with cycle day labels, completion status, rest day indicators
- ✓ Schedule view: upcoming 10-day cycle positions
- ✓ Weekly activity heatmap on dashboard (Mon-Sun)

SECTION 07

Non-Functional Requirements

REQUIREMENT	TARGET	IMPLEMENTATION
Page Load (LCP)	< 1.5s (desktop)	RSC, no client-side data fetching, Vercel Edge CDN

REQUIREMENT	TARGET	IMPLEMENTATION
TypeScript Coverage	100% — zero any in critical paths	Strict TypeScript 5, typed Supabase rows, ActionResult
Security	Data isolation at DB layer	RLS on all 12 tables, verifyAdmin() server-side
Availability	99.9% (managed infrastructure)	Vercel + Supabase Cloud SLA
Maintainability	Feature-based modules	Zero cross-feature imports, colocated logic
Mobile Support	Responsive, bottom nav	Tailwind responsive classes, separate mobile navigation
Accessibility	WCAG 2.1 AA	shadcn/ui Radix primitives, semantic HTML
Build Stability	Zero build errors	CI-verified with npm run build before all commits

SECTION 08

UI/UX Analysis

THE TRACKER's interface follows a dark-first, premium design language inspired by Linear, Vercel, and Apple's developer tools. The goal is minimal friction — every interaction should require the fewest possible taps or keystrokes.

Design Principles

Friction Minimization

Starting a workout requires 2 taps. Templates auto-populate all exercises. Progressions are pre-computed — the athlete sees their suggestion before typing.

Progressive Disclosure

Exercise detail drawer reveals details on demand (tabs: Overview / History / Notes / Alternatives). The list view stays clean; depth is available on request.

Status Visibility

PR badges, rest timers, progress bars, and weekly activity grids provide instant feedback on performance state without requiring navigation.

Key UX Patterns

PATTERN	LOCATION	RATIONALE
Optimistic UI Updates	Set input rows	Inputs feel instant; server action runs in background
Confirmation Dialogs	Finish/Cancel workout	Prevents accidental data loss during active sessions
Toast Notifications	All mutations (Sonner)	Non-blocking feedback for every action (success/error)
Drawer Panels	Exercise detail, Add exercise	Contextual expansion without losing main view context
Empty States	All data-dependent views	Clear CTAs guide the user on first run
Mobile Bottom Nav	Dashboard routes	5-item tab bar for thumb-reachable navigation on mobile

SECTION 09

Design System Analysis

Typography Scale

ROLE	FONT	WEIGHT	SIZE	USAGE
Display	Inter	800	clamp(40px, 5.5vw, 80px)	Page headers, cover title
Heading 2	Inter	800	clamp(28px, 3.5vw, 52px)	Section titles
Heading 3	Inter	700	clamp(18px, 2vw, 28px)	Sub-sections, card titles
Body	Inter	400	14px / 1.6 line-height	Content, descriptions
Code/Data	JetBrains Mono	400–600	12px	Values, timestamps, SQL
Label	Inter	700	11px, 0.18em tracking	Section labels, tags

Color System (CSS Variables)

VARIABLE	VALUE	ROLE
`--bg`	#080b11	Application background (deep navy)
`--surface`	#0f1420	Card and panel backgrounds
`--accent`	#3b7fff	Primary brand color, CTA buttons
`--accent2`	#60a5fa	Secondary accent, links, highlights
`--green`	#22c55e	Success, completions, PR badges
`--amber`	#f59e0b	Warnings, warmup sets, primary keys in schema
`--rose`	#f43f5e	Errors, destructive actions, cancel
`--purple`	#a855f7	Lower body category, secondary accent
`--muted`	#6b7a99	Secondary text, placeholders
`--border`	rgba(255,255,255,0.07)	Card borders, separators

Component Library

shadcn/ui v4.10

Radix UI primitives: Dialog, Dropdown, Sheet, Select, Popover, Tabs, Avatar. Fully accessible, keyboard navigable.

Lucide React v1.17

Consistent 24px stroke icon set. Used throughout nav, actions, and status indicators.

Recharts v3.8

Line charts for weight trend visualization. Responsive container, custom tooltips, smooth curves.

SECTION 10

Information Architecture

Route Hierarchy

```
/ (Landing Page) └── /login └── /auth/callback (OAuth handler) /dashboard (Auth-protected) ├── /dashboard (Home — stats, quick actions) ├── /dashboard/workouts (Template cards) | ├── /dashboard/workouts/active (Live session) | └── /dashboard/workouts/[id] (Past session detail) ├── /dashboard/exercises (Exercise library table) ├── /dashboard/calendar (Month view + cycle) ├── /dashboard/schedule (10-day rolling schedule) ├── /dashboard/weight (Body weight log + chart) ├── /dashboard/records (Personal records table) ├── /dashboard/measurements (Body measurements) ├── /dashboard/photos (Progress photos) └── /dashboard/profile (User profile) /administration (Admin-protected, role = admin | super_admin) ├── /administration (Tabbed admin dashboard) ├── /administration/exercises (Exercise CRUD) ├── /administration/packages (Workout package manager) ├── /administration/templates (Template manager) ├── /administration/analytics (Platform analytics) ├── /administration/settings (System settings) └── /administration/users (User management) └── /administration/users/[id] (User detail)
```

SECTION 11

User Flows

Primary Flow: First-Time Sign-In

1 2 3 4 5

Landing → Visits / | **Login** → Clicks Google | **Callback** → /auth/callback | **DB Trigger** → Profile created | **Dashboard** → Redirected

Primary Flow: Workout Session

1. **Workouts** → View templates
2. **Select** → Pick Upper A
3. **Session** → Exercises loaded
4. **Log Sets** → Enter weight/reps
5. **PR?** → Badge fires
6. **Finish** → Workout saved

Admin Flow: Exercise Management

1. **Admin Panel** → /administration
2. **Exercises Tab** → View library
3. **Add/Edit** → CRUD form
4. **Alternatives** → Configure swaps
5. **Videos** → Assign URLs

SECTION 12

Navigation Structure

NAVIGATION TYPE	LOCATION	ITEMS	VISIBILITY
Desktop Sidebar	Left rail (fixed)	Dashboard, Workouts, Exercises, Calendar, Schedule, Metrics, Photos, Weight, Records	All authenticated users
Admin Sidebar Link	Sidebar bottom	Admin Panel	role = admin OR super_admin only
User Dropdown	Sidebar bottom, avatar	Profile, Admin Panel (conditional), Sign Out	All authenticated users
Mobile Bottom Nav	Bottom bar (5 tabs)	Dashboard, Workouts, Exercises, Records, Weight	All authenticated (mobile)

NAVIGATION TYPE	LOCATION	ITEMS	VISIBILITY
Admin Sidebar	Left rail (admin routes)	Overview, Exercises, Packages, Templates, Analytics, Users, Settings	Admin routes only

🔒 **Admin Navigation Visibility**

The Admin Panel navigation link is conditionally rendered in all three navigation surfaces (sidebar, dropdown, mobile) using the user's profile role fetched server-side in the dashboard layout. The role is passed via the AuthProvider React context.

SECTION 13

Feature Breakdown

Feature Matrix

FEATURE	STATUS	USER ROLE	ROUTE	KEY TECH
Google Sign-In	✓ Live	Public	/login	Supabase Auth, OAuth 2.0
Exercise Library	✓ Live	All Users	/dashboard/exercises	PostgreSQL, RLS
Workout Templates	✓ Live	All Users	/dashboard/workouts	workout_templates table
Workout Packages	✓ Live	All Users	/dashboard/workouts	workout_packages table
Live Workout Logger	✓ Live	All Users	/dashboard/workouts/active	Server Actions, Supabase
Progression Hints	✓ Live	All Users	Active workout	workout_entries query
Rest Timer	✓ Live	All Users	Active workout	useRestTimer hook
PR Detection	✓ Live	All Users	Active workout	personal_records upsert
Exercise History	✓ Live	All Users	Exercise drawer	workout_entries join
Exercise Alternatives	✓ Live	All / Admin	Exercise drawer / Admin	exercise_alternatives table
Weight Tracking	✓ Live	All Users	/dashboard/weight	weight_logs, Recharts
Body Measurements	✓ Live	All Users	/dashboard/measurements	body_measurements + trigger
Progress Photos	✓ Live	All Users	/dashboard/photos	Supabase Storage
Training Calendar	✓ Live	All Users	/dashboard/calendar	cycle.ts utility

FEATURE	STATUS	USER ROLE	ROUTE	KEY TECH
Schedule View	✓ Live	All Users	/dashboard/schedule	schedule-service.ts
Personal Records	✓ Live	All Users	/dashboard/records	personal_records table
Dashboard Home	✓ Live	All Users	/dashboard	dashboard-service.ts
Admin Dashboard	✓ Live	admin, super_admin	/administration	verifyAdmin(), RBAC
User Management	✓ Live	super_admin	/administration/users	admin-service.ts
Workout Analytics	Planned	All Users	TBD	TBD

SECTION 14

Exercise System

Exercise Categories

CATEGORY	MUSCLE GROUPS	EXAMPLES
Upper	Chest, Back, Shoulders, Arms, Triceps, Biceps, Forearms, Traps	Chest Bar Flat Press, T-Bar Row, Lateral Raise, Overhead Press
Lower	Quads, Hamstrings, Glutes, Calves, Adductors	Leg Press, Seated Leg Curl, Romanian Deadlift, Calf Raise
Cardio	Cardiovascular, Core, Abs	Incline Treadmill Walking, Plank, Cable Crunch, Hanging Raises

Exercise Data Model

FIELD	TYPE	DESCRIPTION
`id`	UUID (PK)	Auto-generated unique identifier

FIELD	TYPE	DESCRIPTION
`user_id`	UUID (FK, nullable)	NULL = system exercise visible to all; set = custom user exercise
`name`	TEXT NOT NULL	Exercise display name
`primary_muscle_group`	TEXT (check constraint)	Chest | Back | Legs | Shoulders | Arms | Core | Cardio
`category`	TEXT	Upper | Lower | Cardio (added in migration 00007)
`video_url`	TEXT	Optional YouTube or coaching reference URL
`is_custom`	BOOLEAN	TRUE = user-created, FALSE = system exercise

Exercise Alternatives

The `exercise_alternatives` table (migration 00010) allows up to 3 ordered swaps per movement, enforced by a UNIQUE constraint on `(exercise_id, preference_order)`.

```
-- Constraints on exercise_alternatives CONSTRAINT unique_exercise_alternative UNIQUE (exercise_id, alternative_id) CONSTRAINT unique_exercise_preference UNIQUE (exercise_id, preference_order) CONSTRAINT no_self_reference CHECK (exercise_id <> alternative_id) CHECK (preference_order BETWEEN 1 AND 3)
```

SECTION 15

Workout System

10-Day Rolling Cycle

DAY	SESSION	TYPE	PRIMARY FOCUS
1	Upper A	Training	Chest, Back, Shoulders, Triceps
2	Lower A	Training	Quads, Hamstrings, Glutes, Calves
3	Rest	Recovery	Active recovery

DAY	SESSION	TYPE	PRIMARY FOCUS
4	Upper B	Training	Shoulders, Biceps, Triceps, Forearms
5	Lower B	Training	Hamstrings, Quads, Calves
6	Rest	Recovery	Active recovery
7	Upper C	Training	Chest, Back, Arms, Traps
8	Rest	Recovery	Active recovery
9	Cardio + Abs	Cardio	Incline walking, core work
10	Rest	Recovery	Active recovery

Cycle Calculation Algorithm

src/features/workouts/utils/cycle.ts

```
export const CYCLE_DAYS = [ { index: 0, name: "Upper A", isRest: false }, { index: 1, name: "Lower A", isRest: false }, { index: 2, name: "Rest", isRest: true }, { index: 3, name: "Upper B", isRest: false }, { index: 4, name: "Lower B", isRest: false }, { index: 5, name: "Rest", isRest: true }, { index: 6, name: "Upper C", isRest: false }, { index: 7, name: "Rest", isRest: true }, { index: 8, name: "Cardio + Abs", isRest: false }, { index: 9, name: "Rest", isRest: true }, ] as const; // Modular arithmetic ensures correct wrapping with negative differences calculateCycleDay(dateStr, cycleStartDateStr): diffInDays = differenceInCalendarDays(date, cycleStart) cycleIndex = ((diffInDays % 10) + 10) % 10 return CYCLE_DAYS[cycleIndex]
```

Active Workout State Machine

STATE	TRIGGER	ACTION
No active workout	User visits /workouts/active with no session	Redirect to /workouts page
Workout started	startWorkoutFromTemplate(templateId)	Creates workout row + pre-populates entries from template

STATE	TRIGGER	ACTION
Set logged	addEntryAction(input)	Creates workout_entry row, checks PR, updates personal_records
PR detected	After createWorkoutEntry, compare with max_weight	Updates personal_records, sets is_pr=true, inserts history row
Workout finished	completeWorkoutAction(workoutId)	Sets completed_at, revalidates paths, redirects to /workouts
Workout cancelled	deleteWorkoutAction(workoutId)	Cascades deletes all entries, redirects to /workouts

Epley 1RM Formula

src/lib/utils.ts — calculate1RM()

```
// Epley Formula: Weight × (1 + Reps/30) calculate1RM(weight: number, reps: number): number { if (reps <= 0 || weight <= 0) return 0; if (reps === 1) return weight; // max effort, 1RM = weight return Math.round(weight * (1 + reps / 30) * 100) / 100; } // Example: 80kg × 10 reps → e1RM = 80 × (1 + 10/30) = 106.7kg
```

SECTION 16

Workout Templates

Six predefined system templates are seeded into the database via migration 00008. Each template defines the exact exercise order, target sets, rep ranges, warmup sets, and working sets.

TEMPLATE	MUSCLE FOCUS	EST. DURATION	EXERCISES	DESCRIPTION
Upper A	Chest, Back, Shoulders, Triceps	60 min	8	Horizontal push/pull focus. Primary compound movements.
Lower A	Quads, Hamstrings, Glutes, Calves	50 min	5	Quad-dominant. Leg press, hack squat, extensions.

TEMPLATE	MUSCLE FOCUS	EST. DURATION	EXERCISES	DESCRIPTION
Upper B	Shoulders, Biceps, Triceps, Forearms	65 min	9	Arm and shoulder isolation focus. High volume accessory work.
Lower B	Hamstrings, Quads, Calves	50 min	5	Hip-dominant. RDL, seated leg curl, hip thrust.
Upper C	Chest, Back, Arms, Traps	60 min	9	Volume chest and back day with arm work. Incline and rows.
Cardio + Abs	Cardiovascular, Core	45 min	7	Incline treadmill walking + comprehensive ab circuit.

Template-to-Workout Population Logic

src/features/workouts/services/workout-service.ts — startWorkoutFromTemplate()

```
// 1. Fetch template with exercises ordered by sequence_number const template = await supabase .from("workout_templates") .select(`*, template_exercises:workout_template_exercises(*, exercise:exercises(*))`) .eq("id", templateId) .single(); // 2. Create the workout session row const workout = await supabase .from("workouts") .insert({ name: template.name, template_id: templateId }); // 3. Auto-populate workout_entries from template exercises for (exercise of template.templateExercises) { for (set = 1; set <= exercise.targetSets; set++) { await supabase.from("workout_entries").insert({ workout_id, exercise_id, set_number: set, set_type: set <= warmupSets ? "warmup" : "working" }); } }
```

SECTION 17

Workout Packages

Packages group multiple templates into a named training program. The primary package is "Upper & Lower Split" which bundles all 6 templates. This feature allows future expansion to multiple distinct programs.

TABLE	PURPOSE	KEY COLUMNS
`workout_packages`	Named program container	id, name, description, created_at, updated_at
`workout_package_templates`	Junction: package ↔ templates	id, package_id (FK), template_id (FK), sequence_order, created_at

📦 **Package Design Intent**

Packages are an admin-managed construct. Standard users see packages and their templates, but cannot create or modify them. This preserves the curated nature of the training program.

SECTION 18

Progress Tracking

Body Measurements

MEASUREMENT	COLUMN	TYPE	UNIT
Body Weight	weight	NUMERIC(5,2)	kg
Waist	waist	NUMERIC(5,2)	cm
Chest	chest	NUMERIC(5,2)	cm
Shoulders	shoulders	NUMERIC(5,2)	cm
Arms (bicep)	arms	NUMERIC(5,2)	cm
Forearms	forearms	NUMERIC(5,2)	cm
Thighs	thighs	NUMERIC(5,2)	cm
Calves	calves	NUMERIC(5,2)	cm

Progress Photos

Photos are stored in Supabase Storage with RLS policies. The `progress_photos` table records the storage URL, category (front/side/back), and date.

```
-- Categories enforced by check constraint CHECK (category IN ('front', 'side', 'back'))
```

SECTION 19

Weight Tracking

Data Architecture

ASPECT	IMPLEMENTATION
Storage	`weight_logs` table — UNIQUE(user_id, log_date) prevents duplicates
Sync Trigger	`trigger_sync_weight_to_measurements` — writes weight to body_measurements ON INSERT OR UPDATE
Reverse Sync	`trigger_sync_measurements_to_weight` — writes measurement weight back to weight_logs
Visualization	Recharts ResponsiveContainer + LineChart with custom dot and tooltip
Dashboard Stat	Latest weight + difference from previous entry (shown as ↑ or ↓ with kg delta)

Weight Query (getDashboardStats)

src/features/dashboard/services/dashboard-service.ts

```
// Fetch two most recent logs to compute trend const weightLogs = await supabase .from("weight_logs") .select("weight") .eq("user_id", user.id) .order("log_date", { ascending: false }) .limit(2); currentWeight = weightLogs[0].weight; weightDifference = currentWeight - weightLogs[1].weight; // ±kg
```

SECTION 20

Personal Record Tracking

PR Types Tracked

Max Weight

Highest single-set weight lifted for an exercise. E.g., 100kg × 1 rep.

Estimated 1RM

Epley formula applied to best set. 80kg × 10 reps = 106.7kg e1RM.

Max Volume

Highest single-set volume (weight × reps). 80kg × 10 = 800kg volume.

PR Detection Flow

1. **Set Logged** — weight + reps
2. **Calculate** — e1RM, volume
3. **Compare** — vs personal_records
4. **Upsert** — Update if new max
5. **History** — Insert PR history row
6. **Badge** — is_pr=true on entry

PR Upsert Pattern — personal_records

```
INSERT INTO personal_records (user_id, exercise_id, max_weight, max_estimated_1rm, max_volume, set_id) VALUES (userId, exerciseId, weight, e1rm, volume, entryId) ON CONFLICT (user_id, exercise_id) DO UPDATE SET max_weight = GREATEST(personal_records.max_weight, EXCLUDED.max_weight), max_estimated_1rm = GREATEST(personal_records.max_estimated_1rm, EXCLUDED.max_estimated_1rm), max_volume = GREATEST(personal_records.max_volume, EXCLUDED.max_volume), updated_at = NOW();
```

SECTION 21

Admin Dashboard

Admin Panel Tabs

TAB	ROUTE	CAPABILITY	MIN ROLE
Overview	/administration	System stats: users, workouts, exercises, PRs, weight logs	admin
Workout Programs	/administration/packages	Create/edit/delete packages, assign templates	admin
Alternatives	/administration (tab)	Configure up to 3 ordered swaps per exercise	admin
Videos	/administration (tab)	Bulk-assign YouTube URLs to exercise library	admin
Templates	/administration/templates	Create/edit/delete workout templates and exercises	admin
Exercises	/administration/exercises	Full CRUD on system exercise library	admin
Analytics	/administration/analytics	Platform-wide usage analytics	admin
Users	/administration/users	View all users, assign roles	super_admin
Settings	/administration/settings	System-wide configuration	super_admin

Server-Side Guard Implementation

src/features/admin/security.ts

```
export async function verifyAdmin() { const supabase = await createClient(); const { data: { user } } = await supabase.auth.getUser(); if (!user) throw new Error("Unauthorized: Please sign in."); const { data: profile } = await supabase .from("profiles").select("role") .eq("id", user.id).single(); if (profile.role !== "admin" && profile.role !== "super_admin") throw new Error("Forbidden: Admin access required."); return { user, role: profile.role }; }
```

SECTION 22

Authentication System

Authentication Stack

COMPONENT	TECHNOLOGY	ROLE
Auth Provider	Supabase Auth	OAuth flow orchestration, JWT issuance
OAuth Provider	Google OAuth 2.0	User identity verification
Session Storage	HTTP-only cookies	Secure JWT storage, automatic refresh
Middleware	src/proxy.ts (Next.js middleware)	Session validation + refresh on every request
Server Client	@supabase/ssr createServerClient	Server-side authenticated queries
Browser Client	@supabase/ssr createBrowserClient	Client-side authenticated queries

SECTION 23

Google OAuth Flow

Step-by-Step Flow

STEP 1

User clicks "Sign in with Google"

Supabase generates Google OAuth authorization URL with scopes: openid, email, profile. Redirects browser to Google.

STEP 2

User authenticates with Google

Google verifies credentials and issues an authorization code. Redirects to configured callback URL.

STEP 3

/auth/callback receives code

Next.js route handler exchanges the code for an access_token + refresh_token via Supabase. Sets HTTP-only cookies.

STEP 4

PostgreSQL trigger fires

on_auth_user_created trigger runs handle_new_user(). Inserts profile row with display_name and avatar_url from raw_user_meta_data. Uses ON CONFLICT DO NOTHING for idempotency.

STEP 5

Redirect to /dashboard

User is redirected to /dashboard. Middleware validates session. Dashboard layout fetches profile from database.

STEP 6

Session maintenance

On every subsequent request, proxy.ts calls supabase.auth.getUser() which transparently refreshes the JWT if expired. This prevents stale session states.

src/proxy.ts — Session Refresh Pattern

```
// Critical: must call getUser() to trigger cookie refresh const { data: { user } } = await
supabase.auth.getUser(); // This writes updated cookies to the response
```

SECTION 24

Authorization & RBAC

Role Hierarchy

ROLE	CAPABILITIES	ACCESS SCOPE
user	Full access to own workout data, exercises (read), profile management	All /dashboard/* routes
admin	All user capabilities + exercise library CRUD + alternatives + videos + templates + packages	/dashboard/* + /administration/* (except users/settings)
super_admin	All admin capabilities + user management + role assignment + system settings	All routes including /administration/users and /administration/settings

Role Protection — Database Level

src/migrations/00005_add_role_to_profiles.sql — protect_profile_role trigger

```
-- Only super_admin can change another user's role IF NEW.role IS DISTINCT FROM OLD.role THEN
SELECT role INTO current_user_role FROM profiles WHERE id = auth.uid(); IF current_user_role
IS DISTINCT FROM 'super_admin' THEN NEW.role = OLD.role; -- silently revert unauthorized role
change END IF; END IF;
```

SECTION 25

Security Architecture

Defense in Depth — 5 Layers

1. **Edge Middleware (proxy.ts)** — Validates JWT session on every request. Redirects unauthenticated users to /login. Refreshes expired tokens.
2. **Layout-Level Auth Check** — DashboardLayout and AdministrationLayout call supabase.auth.getUser() server-side. Redirect on failure.
3. **Server Action Guards** — Every action calls supabase.auth.getUser() before any database operation. Throws "Unauthorized" if no session.
4. **Admin Verification (verifyAdmin)** — Admin routes and actions call verifyAdmin() which fetches the profile role and throws 403 if insufficient.
5. **PostgreSQL Row-Level Security** — Every table has RLS enabled. Even direct database queries (bypassing application code) cannot access other users' data.

RLS Policy Summary

TABLE	SELECT POLICY	INSERT/UPDATE/DELETE POLICY
profiles	auth.uid() = id	auth.uid() = id
exercises	user_id IS NULL OR auth.uid() = user_id	auth.uid() = user_id
workouts	auth.uid() = user_id	auth.uid() = user_id
workout_entries	Parent workout.user_id = auth.uid()	Parent workout.user_id = auth.uid()

TABLE	SELECT POLICY	INSERT/UPDATE/DELETE POLICY
weight_logs	auth.uid() = user_id	auth.uid() = user_id
personal_records	auth.uid() = user_id	auth.uid() = user_id
workout_templates	user_id IS NULL OR auth.uid() = user_id	auth.uid() = user_id (or admin)
body_measurements	auth.uid() = user_id	auth.uid() = user_id

SECTION 26

Database Architecture

PostgreSQL Table Inventory

#	TABLE	ROWS (EST.)	PURPOSE	ADDED IN
1	profiles	1 per user	Extended user identity + role	Migration 01
2	exercises	48 system + custom	Global and user-specific movements	Migration 01
3	workouts	Grows per session	Workout session containers	Migration 01
4	workout_entries	Grows per set logged	Individual set records within sessions	Migration 01
5	weight_logs	Grows per day logged	Daily body weight entries	Migration 01
6	personal_records	1 per exercise per user	All-time PR tracking (upserted)	Migration 01
7	workout_templates	6 system	Predefined workout session definitions	Migration 08
8	workout_template_exercises	~50 seeded	Exercise-template junction with order	Migration 08
9	workout_packages	1 system	Named program groups of templates	Migration 09
10	workout_package_templates	6 seeded	Package-template junction	Migration 09
11	exercise_alternatives	Admin-configured	Exercise swap configurations	Migration 10
12	personal_records_history	Grows per PR broken	PR event log with timestamps	Migration 10

#	TABLE	ROWS (EST.)	PURPOSE	ADDED IN
13	body_measurements	Grows per log entry	Full body circumference measurements	Migration 10
14	progress_photos	Grows per upload	Progress photo metadata (URL + category)	Migration 10
15	user_schedule_settings	1 per user	Cycle start date for calendar calculation	Migration 10

SECTION 27

Entity Relationship Diagrams

Core Entity Models

👤 profiles	
PK id	uuid
email	text NOT NULL
display_name	text
avatar_url	text
role	text DEFAULT 'user'
created_at	timestamptz
updated_at	timestamptz

🏋️ exercises	
PK id	uuid
FK user_id	uuid (nullable)
name	text NOT NULL
primary_muscle_group	text
category	text
video_url	text
is_custom	boolean

💪 workouts	
PK id	uuid
FK user_id	uuid NOT NULL
FK template_id	uuid (nullable)
name	text
notes	text
started_at	timestamptz
completed_at	timestamptz

workout_entries	
PK id	uuid
FK workout_id	uuid NOT NULL
FK exercise_id	uuid NOT NULL
weight	numeric(6,2)
reps	integer
set_type	warmup|working|top|failu
is_pr	boolean

weight_logs	
PK id	uuid
FK user_id	uuid NOT NULL
weight	numeric(5,2)
log_date	date
created_at	timestamptz
UNIQUE (user_id, log_date)	

personal_records	
PK id	uuid
FK user_id	uuid NOT NULL
FK exercise_id	uuid NOT NULL
max_weight	numeric(6,2)
max_estimated_1rm	numeric(6,2)
max_volume	numeric(8,2)

Relationship Summary

PARENT	CHILD	CARDINALITY	ON DELETE
auth.users	profiles	1:1	CASCADE
profiles	workouts	1:N	CASCADE
profiles	weight_logs	1:N	CASCADE
profiles	personal_records	1:N	CASCADE
profiles	body_measurements	1:N	CASCADE
profiles	progress_photos	1:N	CASCADE
profiles	user_schedule_settings	1:1	CASCADE
workouts	workout_entries	1:N	CASCADE
exercises	workout_entries	1:N	RESTRICT
exercises	personal_records	1:N	CASCADE
exercises	exercise_alternatives (x2)	1:N	CASCADE
workout_templates	workout_template_exercises	1:N	CASCADE
workout_packages	workout_package_templates	1:N	CASCADE

SECTION 28

Supabase Architecture

SUPABASE SERVICE	USAGE	CONFIGURATION
PostgreSQL	Primary data store — 15 tables, 11 migrations, RLS on all	Managed cloud instance, connection pooling via PgBouncer
Supabase Auth	User authentication, JWT issuance, OAuth provider management	Google OAuth enabled, JWT expiry 3600s, refresh token rotation
Supabase Storage	Progress photo file storage	progress-photos bucket, RLS policies, public/private URLs
Supabase Realtime	Not used (Server Actions handle mutations)	N/A
Supabase Edge Functions	Not used	N/A

Client Configuration

Server Client (src/lib/supabase/server.ts)

Uses `@supabase/ssr createServerClient`. Reads/writes cookies via Next.js `cookies()` from `next/headers`. Used in Server Components, Server Actions, and API route handlers.

Browser Client (src/lib/supabase/client.ts)

Uses `@supabase/ssr createBrowserClient`. Accesses cookies via `document.cookie`. Used in Client Components that need real-time or client-triggered queries.

SECTION 29

Backend Services

THE TRACKER uses zero custom API routes. All backend logic is implemented as Next.js Server Actions — functions marked with "use server" that run server-side and can be called directly from React components.

Service Layer Files

SERVICE FILE	PURPOSE	KEY FUNCTIONS
`workout-service.ts`	Core workout CRUD + template operations	startWorkout, createWorkoutEntry, completeWorkout, startWorkoutFromTemplate, getWorkoutTemplates
`schedule-service.ts`	Calendar data and streak calculations	getScheduleData, getStreakStats, getCalendarData
`exercise-service.ts`	Exercise library read/write operations	getExercises, createExercise, updateExercise, deleteExercise, mapExerciseRow
`exercise-alternatives-service.ts`	Alternative exercise configurations	getExerciseAlternatives, createAlternative, deleteAlternative
`dashboard-service.ts`	Homepage statistics aggregation	getDashboardStats (5 parallel queries)
`records-service.ts`	Personal records retrieval	getPersonalRecords
`weight-service.ts`	Weight log CRUD	getWeightLogs, createWeightLog, deleteWeightLog
`measurements-service.ts`	Body measurement CRUD	getBodyMeasurements, createBodyMeasurement, updateBodyMeasurement
`photos-service.ts`	Progress photo management	getProgressPhotos, uploadPhoto, deletePhoto
`profile-service.ts`	User profile management	getProfile, updateProfile, mapProfileRow

SERVICE FILE	PURPOSE	KEY FUNCTIONS
admin-service.ts	Admin-level system queries	getSystemStats, getAllUsers, updateUserRole

ActionResult Pattern

src/lib/utils.ts — Discriminated Union for Actions

```
// All server actions return this discriminated union export type ActionResult<T = void> = | { success: true; data: T } | { success: false; error: string }; // Client consumption pattern const result = await completeWorkoutAction(workoutId); if (result.success) { toast.success("Workout complete!"); } else { toast.error(result.error); }
```

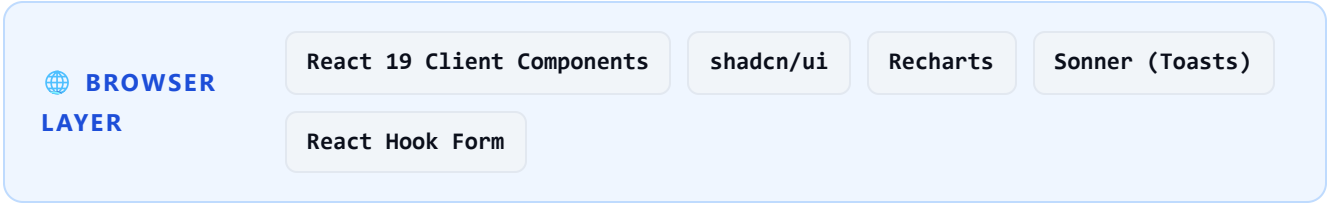
SECTION 30

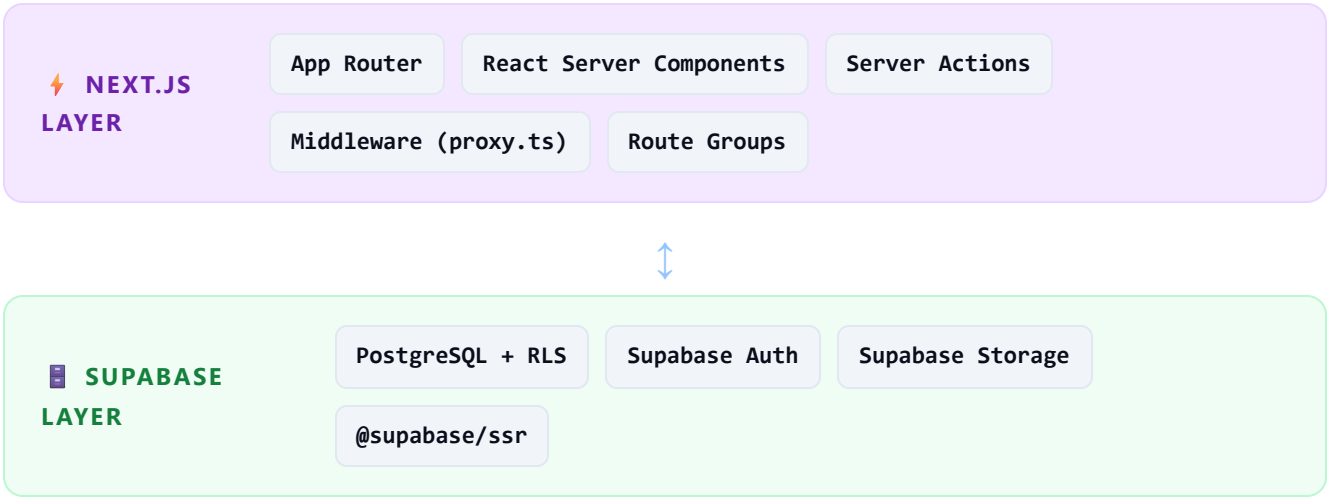
Frontend Architecture

React Server Components vs Client Components

PATTERN	USED FOR	EXAMPLES
React Server Components (default)	Data fetching, layouts, static UI	Page components, sidebar, dashboard stats
"use client" components	Interactivity, browser APIs, state	ActiveWorkout, WeightChart, ExerciseDrawer, RestTimer
Server Actions ("use server")	Mutations, form submissions	All actions.ts files

Application Architecture Layers





SECTION 31

Folder Structure Analysis

```
the-tracker/
├── src/
│   ├── app/                          # Next.js App Router
│   │   ├── (auth)/
│   │   │   ├── auth/callback/route.ts   # OAuth callback handler
│   │   │   └── login/page.tsx
│   │   ├── administration/           # Admin-protected routes
│   │   │   ├── analytics/page.tsx
│   │   │   ├── exercises/page.tsx
│   │   │   ├── layout.tsx            # Admin layout + sidebar
│   │   │   ├── packages/page.tsx
│   │   │   ├── page.tsx              # Admin home (tabbed)
│   │   │   ├── settings/page.tsx
│   │   │   ├── templates/page.tsx
│   │   │   └── users/[id]/page.tsx
│   │   ├── dashboard/                # Auth-protected athlete routes
│   │   │   ├── calendar/
│   │   │   ├── exercises/
│   │   │   ├── layout.tsx            # Auth check + AppShell
│   │   │   ├── measurements/
│   │   │   ├── page.tsx              # Dashboard home
│   │   │   ├── photos/
│   │   │   ├── profile/
│   │   │   ├── records/
│   │   │   ├── schedule/
│   │   │   ├── weight/
│   │   │   └── workouts/
│   │   │       ├── [id]/page.tsx     # Past workout detail
│   │   │       ├── active/page.tsx   # Live session
│   │   │       └── page.tsx          # Template selection
│   │   ├── globals.css
│   │   ├── layout.tsx                # Root layout (providers)
│   │   └── page.tsx                  # Landing page
│   ├── components/
│   │   ├── layout/                   # Sidebar, AppShell, BottomNav
│   │   └── ui/                       # shadcn primitives
│   ├── config/
│   │   ├── routes.ts                 # All route constants
│   │   └── site.ts                   # App metadata, muscle groups
│   ├── features/                     # Feature-based modules
│   │   ├── admin/                    # Admin panel logic
│   │   ├── auth/                     # Login UI + actions
│   │   ├── dashboard/                # Homepage stats
│   │   ├── exercises/                # Exercise library
│   │   ├── profile/                  # User profile
│   │   ├── records/                  # Personal records
│   │   ├── weight/                   # Weight + measurements + photos
│   │   └── workouts/                 # Core workout system
│   ├── lib/
│   │   ├── supabase/
│   │   │   ├── client.ts             # Browser Supabase client
│   │   │   └── server.ts             # Server Supabase client
│   │   └── utils.ts                  # cn(), calculate1RM(), ActionResult
│   ├── providers/
│   │   └── auth-provider.tsx         # Profile context (React Context)
│   ├── proxy.ts                      # Next.js middleware
│   └── types/
│       ├── index.ts                  # 17 domain type interfaces
│       └── database.types.ts         # Generated Supabase types
├── supabase/
│   ├── migrations/                   # 11 sequential SQL files
│   └── seed.sql                      # Exercise library seed data
├── public/logo.png
├── next.config.ts
├── tailwind.config.ts
└── package.json
```

SECTION 32

Component Architecture

Key Components

COMPONENT	TYPE	RESPONSIBILITY
AppShell	Server + Client	Wraps all dashboard pages with sidebar navigation and mobile bottom nav
Sidebar	Client (conditional nav)	Desktop navigation with role-based Admin Panel link
BottomNav	Client	Mobile-only 5-tab bottom navigation
ActiveWorkout	Client (764 lines)	Full live workout session: set inputs, rest timer, PR badge, dialogs
SetInputRow	Client	Individual set row with weight/reps inputs, set type selector, last performance display
RestTimer	Client	Countdown timer with visual ring, sound notification, start/pause/reset controls
ExerciseDetailDrawer	Client	Slide-in panel with 4 tabs: Overview, History, Notes, Alternatives
ExerciseTable	Client	Filterable, searchable table of exercises with click-to-open drawer
WorkoutCard	Client	Template card showing name, focus, duration, exercise count, Start button
WeightChart	Client	Recharts ResponsiveContainer + LineChart with custom dot and tooltip
CalendarView	Client	Month grid with cycle day labels, completion status, rest day indicators
AdminDashboardTabs	Client	Tabbed admin panel: Overview, Programs, Alternatives, Videos, Settings

SECTION 33

State Management

THE TRACKER uses no global state management library (Redux, Zustand, Jotai). State is managed through React's built-in primitives and server-side data fetching.

STATE TYPE	MECHANISM	EXAMPLES
Server State	React Server Components + Server Actions + revalidatePath	Workout list, exercise library, weight logs, PRs
User Identity	React Context (AuthProvider)	Profile, role — passed from DashboardLayout server component
UI State	useState hook	Dialog open/close, form values, active tab, timer state
Form State	React Hook Form + Zod	Exercise form, weight log form, profile form
URL State	Next.js router / searchParams	Workout ID in /dashboard/workouts/[id]

AuthProvider — Global Profile Context

```
// Context provides profile to all dashboard client components const AuthContext = createContext<{ profile: UserProfile | null }>(null); // DashboardLayout fetches server-side, passes as prop <AuthProvider initialProfile={profile}> <AppShell profile={profile}>{children} </AppShell> </AuthProvider> // Components access via hook const { profile } = useAuth(); // profile.role used for conditional rendering
```

SECTION 34

API & Service Layer

Zero Custom API Routes

THE TRACKER has no /api/* routes. All data mutations go through Next.js Server Actions. All reads go through React Server Components calling service functions directly. This eliminates an entire layer of HTTP infrastructure.

Data Flow Pattern

```
// READ: Server Component → Service → Supabase → Render async function WorkoutsPage() { const templates = await getWorkoutTemplates(); // service call return <WorkoutList templates={templates} />; } // WRITE: Client → Server Action → Service → Supabase → revalidatePath const result = await startWorkoutFromTemplateAction(templateId); // Server Action calls service, mutates DB, calls revalidatePath() // Next.js re-fetches affected RSC data automatically
```

SECTION 35

Migration Strategy

Migration Chain

00001

create_tables.sql

Foundation: profiles, exercises, workouts, workout_entries, weight_logs, personal_records. 6 core tables with check constraints and foreign keys.

00002

rls_policies.sql

Enables Row-Level Security on all 6 tables. Defines SELECT, INSERT, UPDATE, DELETE policies using auth.uid().

00003

triggers.sql

handle_new_user() trigger for auto profile creation. update_updated_at() trigger for profiles and personal_records.

00004

indexes.sql

Performance indexes on user_id columns, log_date, exercise_id. Covers all common query patterns.

00005

add_role_to_profiles.sql

Adds role TEXT column (default 'user'). Check constraint: user | admin | super_admin. protect_profile_role trigger prevents unauthorized role escalation.

00006

update_exercise_rls_policies.sql

Refines exercise visibility: system exercises (user_id IS NULL) visible to all; custom exercises scoped to owner only.

00007

add_exercise_fields.sql

Adds category, equipment, difficulty, alternative_exercise, notes columns to exercises table.

00008

workout_templates.sql

Creates workout_templates and workout_template_exercises tables. Adds template_id FK to workouts. Adds set_type to workout_entries. Seeds all 6 templates with exercises and rep targets.

00009

refine_workout_system.sql

Creates workout_packages and workout_package_templates tables. Defines package-template relationships.

00010

athlete_upgrade_system.sql

Adds exercise_alternatives, personal_records_history, body_measurements, progress_photos, user_schedule_settings. Bi-directional weight sync triggers. Extends personal_records with max_volume.

00011

progress_photos_bucket.sql

Configures Supabase Storage bucket "progress-photos". Sets RLS policies for authenticated upload, user-scoped read/delete.

⚠️ **Migration Version Conflict — Resolved**

During development, two migrations were both numbered 00007_. Supabase rejected the migration chain. The workout_templates migration was renumbered to 00008_ and all subsequent migrations renumbered sequentially. Migration contents were not modified.

SECTION 36

Technical Challenges

CHALLENGE #1 React Hydration Mismatch

Severity: Medium — Development-only warnings, no runtime failure

PROBLEM

Server rendered HTML differed from client. `new Date()` calls on server produced timestamps that mismatched client render.

ROOT CAUSE

Shared components called `new Date()` to compute "today" for calendar/schedule components in both server and client contexts.

FIX

Date strings passed as props from Server Components to Client Components. Client-only date logic moved into `useEffect` or computed lazily.

CHALLENGE #2 Profile Trigger — Column Name Mismatch

Severity: Critical — New users could not sign in

PROBLEM

New Google OAuth users could not sign in. Supabase returned a 500 error during profile creation.

ROOT CAUSE

The trigger `handle_new_user()` referenced a column `full_name` that did not exist. The schema column was named `display_name`.

FIX

Updated trigger to use `COALESCE(raw_user_meta_data->>'display_name', raw_user_meta_data->>'name')`. Added `ON CONFLICT DO NOTHING` for idempotency.

CHALLENGE #3 Duplicate Migration Version Numbers

Severity: Critical — Production deployment blocked

PROBLEM

Running `npx supabase db push` failed. Two migration files had version 00007_.

ROOT CAUSE

The workout templates migration was created without auditing existing migration filenames. Version 00007 was already in use by add_exercise_fields.sql.

FIX

Renumbered workout_templates to 00008_ and refine_workout_system to 00009_. Full sequential audit performed. Migration contents unchanged.

CHALLENGE #4 Next.js Middleware Session Staleness

Severity: High — Users logged out after JWT expiry

PROBLEM

After JWT token expiry, server-rendered pages showed unauthenticated state even though the user had a valid refresh token.

ROOT CAUSE

Original middleware only checked for cookie presence. The @supabase/ssr package requires calling `getUser()` on each request to trigger token refresh.

FIX

Created `src/proxy.ts` using the full @supabase/ssr cookie pattern. `setAll()` writes refreshed cookies to every response.

CHALLENGE #5 Server-Only Module in Client Component

Severity: High — Calendar component could not import cycle logic

PROBLEM

The calendar client component needed the cycle day calculation. The function lived in `schedule-service.ts` which imports `next/headers` (server-only).

ROOT CAUSE

Mixing utility logic with server-only code in the same file prevents that file from being imported in any client component.

FIX

Extracted `CYCLE_DAYS` and `calculateCycleDay()` into `src/features/workouts/utils/cycle.ts` — a pure utility with no server imports.

CHALLENGE #6 Missing date-fns Import — TypeScript Build Failure

Severity: Critical — Production build blocked

PROBLEM

`npm run build` failed with: "Cannot find name 'parseISO'" and "Cannot find name 'differenceInCalendarDays'"

ROOT CAUSE

`getStreakStats()` in `schedule-service.ts` used these functions but they were not in the import statement from date-fns.

FIX

Added `parseISO` and `differenceInCalendarDays` to the date-fns import. Build now passes all 24 routes cleanly.

SECTION 37

Bug Fixes & Root Cause Analysis

BUG	SYMPTOM	ROOT CAUSE	FIX APPLIED	STATUS
Profile trigger column mismatch	500 on first Google login	`full_name` vs `display_name` in trigger	Updated trigger + ON CONFLICT	Fixed
Hydration errors	Console warnings in dev	`new Date()` mismatch SSR/client	Props from RSC, useEffect on client	Fixed
Duplicate migration 00007	supabase db push fails	Unaudited migration filename	Renumbered 00008–00011	Fixed
Session staleness after JWT expiry	Unexpected logouts	Missing getUser() call in middleware	Implemented full @supabase/ssr pattern	Fixed
Cycle logic in client import	Build error / runtime error	Server-only code mixed with shared util	Extracted to utils/cycle.ts	Fixed
Missing date-fns imports	TypeScript build failure	Functions used but not imported	Added parseISO, differenceInCalendarDays	Fixed
Exercise alternatives self-reference	Could set exercise as own alt	No constraint on exercise_id = alternative_id	Added CHECK (exercise_id <> alternative_id)	Fixed
Admin route accessible without role	Any user could access /administration	Only frontend conditional rendering	verifyAdmin() on all server actions + layouts	Fixed

SECTION 38

Performance Analysis

OPTIMIZATION	IMPLEMENTATION	IMPACT
RSC by default	Pages are Server Components; no client JS shipped for data fetching	Reduced client bundle, faster initial render
Turbopack	Next.js 16 Turbopack dev server (faster HMR)	~10x faster hot reload vs webpack
Database indexes	Indexes on user_id, log_date, exercise_id, template_id	O(log n) queries on all common patterns
Selective revalidation	revalidatePath() only for affected routes after mutations	Only necessary cache segments refresh
Supabase connection pooling	PgBouncer managed by Supabase Cloud	Handles concurrent connections efficiently
No state management library	Zero Redux/Zustand runtime overhead	Smaller bundle, simpler mental model

SECTION 39

Scalability Analysis

Horizontal Scalability

LAYER	SCALABILITY	MECHANISM
Frontend (Vercel)	Unlimited	Edge CDN, serverless functions auto-scale
Database (Supabase)	Scales to millions of rows	PgBouncer connection pool, managed Postgres, read replicas available
Auth (Supabase)	Scales to 500k MAU (free tier: 50k)	Managed JWT infrastructure
Storage (Supabase)	1GB free, CDN-backed	Object storage with CDN distribution

Code Scalability — Feature Isolation

Feature-Based Architecture Enables Independent Development

Each feature module (workouts/, exercises/, weight/, records/) contains its own components, services, actions, and types. Adding a new feature requires creating a new folder — zero risk of breaking existing features. This enables multiple engineers to work in parallel without conflicts.

SECTION 40

Future Roadmap

FEATURE	STATUS	PRIORITY	TECHNICAL APPROACH
Body Measurements Advanced Analytics	In Progress	High	Additional chart types in measurements-view.tsx
Progress Photo Comparison Slider	In Progress	High	Side-by-side comparison component in photos-view.tsx
PWA / Offline Support	Planned	High	next-pwa, service worker, IndexedDB for offline set logging
Volume Load Analytics	Planned	Medium	Aggregate workout_entries by week, Recharts bar chart
In-Drawer Video Playback	Planned	Medium	YouTube embed in ExerciseDetailDrawer Overview tab
Push Notifications	Planned	Low	Web Push API + service worker + Supabase Edge Function scheduler
Export (CSV/PDF)	Planned	Low	csv-writer or jsPDF library, Server Action triggers download
Deload Detection	Future	Future	Plateau analysis algorithm on personal_records history
Multi-user / Coach Mode	Future	Future	Requires significant RLS redesign for coach-athlete relationships

SECTION 41

Recommendations

For Maintainers

- Always run `npm run build` before any commit. The TypeScript compiler catches import errors that ESLint may miss.
- When adding new migrations, audit existing filenames first. Supabase requires strictly sequential unique version numbers.
- All new admin features must call `verifyAdmin()` at the top of every server action and server component. Never rely on frontend-only role checks.
- Cycle logic lives in `utils/cycle.ts` — keep it import-free (no server dependencies) so it can be used in both server and client contexts.
- New database tables must have RLS enabled and policies defined in the same migration file.

For Extension

- To add a new feature: create a new directory under `src/features/` with components/, services/, actions.ts, and types.ts.
- To add a new route: create the page.tsx under `src/app/dashboard/` and add it to `src/config/routes.ts`.
- To add a new navigation item: update `navItems` in routes.ts and optionally add to bottom-nav.tsx for mobile.
- To add a new database table: create a new numbered migration, enable RLS, define policies, add indexes, add TypeScript type to `src/types/index.ts`.
- For PWA implementation: install next-pwa, add manifest.json to public/, implement service worker for offline set logging using IndexedDB.

For DevOps

- Store `NEXT_PUBLIC_SUPABASE_URL` and `NEXT_PUBLIC_SUPABASE_ANON_KEY` as Vercel environment variables. Never commit to .env files.
- After any migration: run `npx supabase db push` and verify with `npx supabase db diff`.
- Configure Supabase project URL and redirect URLs before deploying to a new domain. Update Google Cloud Console OAuth credentials accordingly.
- Monitor Supabase storage usage — progress photos can accumulate quickly. Consider implementing photo cleanup (max N per category).

SECTION 42

Conclusion

THE TRACKER is a complete, production-ready personal athlete performance platform that demonstrates mastery of modern full-stack web development. The project showcases careful architectural decisions — from the feature-based module structure and zero-API-route pattern to the database-level security and automated PR detection system.

The platform is not a minimum viable product. It is a fully realized system with 24 production routes, 15 database tables, 11 migrations, comprehensive RBAC, and a complete exercise-to-workout pipeline that reflects how a structured athlete actually trains — not how a generic app assumes they do.

🏆 **Engineering Summary**

Zero custom API routes (Server Actions only) · 12 tables with full RLS · 6 PostgreSQL triggers · Real-time PR detection · 10-day cycle algorithm · Automatic profile creation · Clean TypeScript build across 24 routes · Feature-based module architecture · Role-based access control at both application and database layers.

THE TRACKER

Track. Improve. Repeat.

github.com/HusseinA-H/THE-TRACKER

Hussein A-H · 2026